

CSCI 1810

10/09/26

Lecture 2

Intro to alignment Algorithms: Global and local:

- since sequences are very long, similarity can often be by chance. We need to denoise and make sure similarity we see is not by chance but due to convergent origin

Global alignment:

Sequence Comparison:

↳ DNA sequences: A, C, G, T

↳ RNA sequences: A, C, G, U

↳ Protein sequences: 20 naturally occurring amino acids

Global ~~alignment~~ similarity, the Basic Similarity Analysis Algorithm:

↳ Scoring ~~schemes~~ schemes

↳ edit graphs

↳ Alignment $\Rightarrow$ a path in the edit graph

↳ The Principle of optimality

↳ The dynamic programming Algorithm

↳

Sequence Alignment:

↳ Input: two sequences over the same alphabet

↳ Output: an alignment of the two sequences

↳ there are 3 possible outcomes for each position in sequence: - match

e.g. - GCGCATG

TGCG--TA

- mismatch

- indel

→ Consider two sequences:

→ $X = x_1, x_2, \dots, x_n$

→ $Y = y_1, y_2, \dots, y_n$

→ Scoring Schemes:

→ Unit score:

for protein, 20 rows

20 columns corresponding to AAs

	A	C	G	T	-
A	1	0	0	0	0
C	0	1	0	0	0
G	0	0	1	0	0
T	0	0	0	1	0
-	0	0	0	0	0

→ $f(x, y)$ = the score for aligning x with y

→ $f(-, y)$ = the score of aligning - with y

→ $f(x, -)$ = the score of aligning y with - with x

→ ~~$f(A, A) + f(-, T) + f(G, C) + f(G, G)$~~ the score of alignment is the sum of the pairwise aligned symbols

→ For proteins, equivalent is Dayhoff score, which has 20 rows & columns, but values are not just 0 & 1, but rather can be positive or negative and diff values depending on similarity of AA characteristics

→ Scoring function: a sum of terms each for a pair of aligned residues, and for each gap

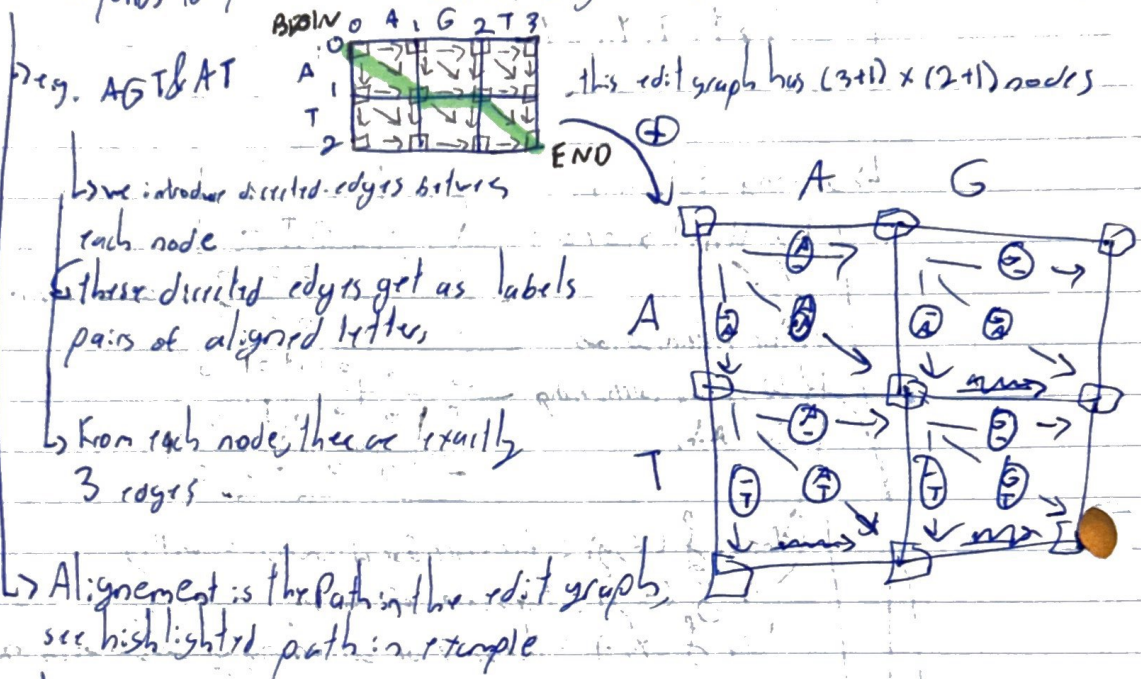
→ is the log of the relative likelihood that the sequences are related compared to being unrelated

→ Identifiers and conservative substitutions are positive terms

→ Non-conserved substitutions are negative terms

The edit graph:

- We treat a graph where alignments between the two sequences corresponds to paths between the begin and end nodes of the graph



- every path from Begin to End corresponds to an alignment and every alignment corresponds to a path between Begin and End

The Principle of Optimality: the optimal answer to a problem is expressed in terms of optimal answer for its sub problems

Dynamic Programming:

- Given two sequences X and Y and a scoring scheme, find an optimal alignment of X with Y (max score alignment)

Part 1: compute the first optimal alignment score

Part 2: construct the optimal alignment

- We are looking for the optimal alignment; the maximal score path in the Edit graph from beginning to end.

(vi) i is ^{column} row number and j is row

↳ The DP Matrix: $\text{Matrix}_s = [sc_{ij}]$

$LS_{SC}(i, j)$ = the score at the minimal cost path from origin vertex to vertex (i, j)

↳ optimal path

e.g. this is $S(1,0)$

the optimal path to (i, j) must be through one of: $-(i-1, j)$

$$= (i, j-1)$$

total optimal path is equal to

$\hookrightarrow$ If it is through $(-1, i)$ it must have opted to $(-1, j) + f(-1, j)$
 Horizontal $\hookrightarrow = sc(-1, j) + f(-1, j)$

Horizontal: $(i, j-1)$ must have optimal path to (i, j)

Diagonal: $(i, j-1)$ total optimal path is equal to $(i-1, j-1) + \text{grid}[i, j]$

Vertical Dividend $\rightarrow (i-1, j-1)$ total optimal path is equal to $(i-1, j-1) + g(x, y, j)$
 $\hookrightarrow s(i-1, j-1) + g(x, y, j)$

the basic algorithm = $\int_{i=1}^n \int_{j=1}^n S(i, j-1) + f(x_i, y_j)$

$$\text{Max } \{s(x_i - l, y) + g(x_{i+1}, y)\}$$

↓
We assign values to nodes by:

2) For every diagonal element, add 1, else for mismatches or indels add 0

- We can use traceback and it tells us that optimal alignment is: A G T
A - T

Local Alignment: focus is two subsequences are close rather than whole sequence

$$s(i, j) = \max \begin{cases} 0, \\ s(i-1, j+1) + g(x_i, y_{j+1}), \\ s(i-1, j) + g(x_i, -), \\ s(i, j-1) + g(-, y_j) \end{cases}$$

General Scoring Schemes:

- Assumptions: 1) Independence of mutations at different sites.
- 2) Gaps of any length are considered one mutation.

- All the efficient alignment algorithms (employing on the dynamic programming method) are based fundamentally on the fact that the scoring function is additive.

Heuristic Interpretation of the scoring scheme:

- Heuristic interpretation of the alignment score:

→ take example $x = \text{AACGTAA}$ & $y = \text{ACGGTGT}$ AACG-TAA
 A-CCGGTGT

→ In this alignment we have: 4 matches, 2 gaps, 2 mismatches

→ Let p be the probability of a match

→ let q be the prob. of a substitution/mismatch

→ let r be the prob. of a single ~~not~~ letter indel/gap

→ the probability of this example alignment is: $P_{\text{alignment}} = p^4 q^2 r^2$

→ $= 4 \log p + 2 \log q + 2 \log r$

↳ the log likelihood of this alignment is:

$$S' = \log(P_{\text{alignment}}) = 4 \log p + 2 \log q + 2 \log r$$

$$S = S' - 8 \log K, K = \text{constant}$$

$$= -4 \log K - 2 \log K - 2 \log K$$

constant does not change anything with the maximisation point, so it is ok to add it for some mathematical manipulation

$$S = S' - 8 \log K = 4 \log p + 2 \log q + 2 \log r - 8 \log K = 4 \log \left(\frac{p}{K} \right) + 2 \log \left(\frac{q}{K} \right) + 2 \log \left(\frac{r}{K} \right)$$

this step requires

$$\Rightarrow \log \left(\frac{p}{K} \right) = 1$$

$$= -4 - 2\mu - 2\delta, \text{ where: } \mu = \log \left(\frac{K}{q} \right) \quad \delta = \log \left(\frac{K}{r} \right)$$

$$\text{choose } K \text{ such that } \log \left(\frac{p}{K} \right) = 1$$

We have 4 common identities

- μ is penalty for mismatch

- δ is penalty for indel

then get penalty μ for ~~penalty~~ mismatch

then get penalty δ for gap/indel